

House Price Prediction

Abstract

Predicting house prices is a critical task for investors and homeowners alike. This report presents a fundamental regression problem in **Machine Learning** for predicting real estate prices by comparative analysis of 18 distinct features using a dataset of 50,000 records. It explores multiple modeling techniques - ranging from **Decision Trees** and **Linear Regression** to **Multi-Layer Perceptrons** - to estimate house prices. Uses **Backpropagation** and **Gradient Descent** to develop a neural network capable of capturing complex relations between the features. The proposed methodology involves preprocessing, feature scaling, and dimensional analysis. Experimental results demonstrate that while the dataset exhibits strong linear characteristics with $R^2 \approx 0.998$, the **Artificial Neural Network (ANN)** provides a strong framework for generalization through Regularization and Gradient Descent optimization.

Introduction

The real estate market is among the most active areas of economics, demonstrating volatility and numerous interrelations between factors. Therefore, prediction of house prices involves consideration of many factors/features of physical characteristics, geographical advantage, and environmental safety. Appraisal procedures are generally complicated, making humans involved in this process likely to make mistakes and causing the need for automation and data analysis.

We examine a predictive system for housing prices based on a dataset comprising **50,000 records**. The central part of this research is built upon Supervised Learning with a Regression problem statement.

We use a variety of algorithms from Linear Regression serving as a baseline model to **Multi-layer Perceptron (MLP)**. We particularly concentrate on the ANN design with the Backpropagation algorithm utilized to improve generalization through Gradient Descent optimization.

Moreover, we discuss how to achieve the generalization in the form of comparison with Decision Trees and Regularization.

Main Objectives

- To develop a high-accurate regression model to predict house prices based on physical attributes, environmental factors and location-based indicators.
- To implement and evaluate an **Artificial Neural Network (ANN)** using the **Backpropagation Algorithm** and compare its performance with baseline models.

- To analyze the impact of **Activation Functions** and Hidden Neurons on the model's ability to minimize prediction error.
- To assess the role of **Regularization** (L2) in improving the model's **Generalization** to unseen data.

Literature Review

Existing research on house price prediction primarily explores the transition from traditional statistical models to advanced machine learning (ML) ensembles to handle the complex, non-linear nature of real estate markets.

Property valuation has traditionally relied on **hedonic** pricing models (**Linear Regression**), posits that a property's value is the sum of its individual attributes like size, age, and location. While it is great for identifying the features which affect price level mostly, these models often struggle with complex non-linearities. Similar case with Multiple Linear Regression also, which struggles when handling high-dimensional data.

Researchers often utilize **Decision Trees** for their interpretability and **K-Nearest Neighbors** (KNN) for similarity-based estimation. Advanced studies highlight that **Multi-Layer Perceptrons (MLPs)** with **Gradient Descent** optimization can outperform traditional methods when dealing with high-dimensional data. **Random Forest** model is far superior to linear models due to its ability to handle both continuous and categorical data and its resistance to overfitting. Some researchers report error rates of less than 2% when using the model.

*source: <https://www.mdpi.com/2413-8851/9/2/32#:~:text=It%20is%20one%20of%20the,than%20traditional%20methodologies%20%5B12%5D>, Google

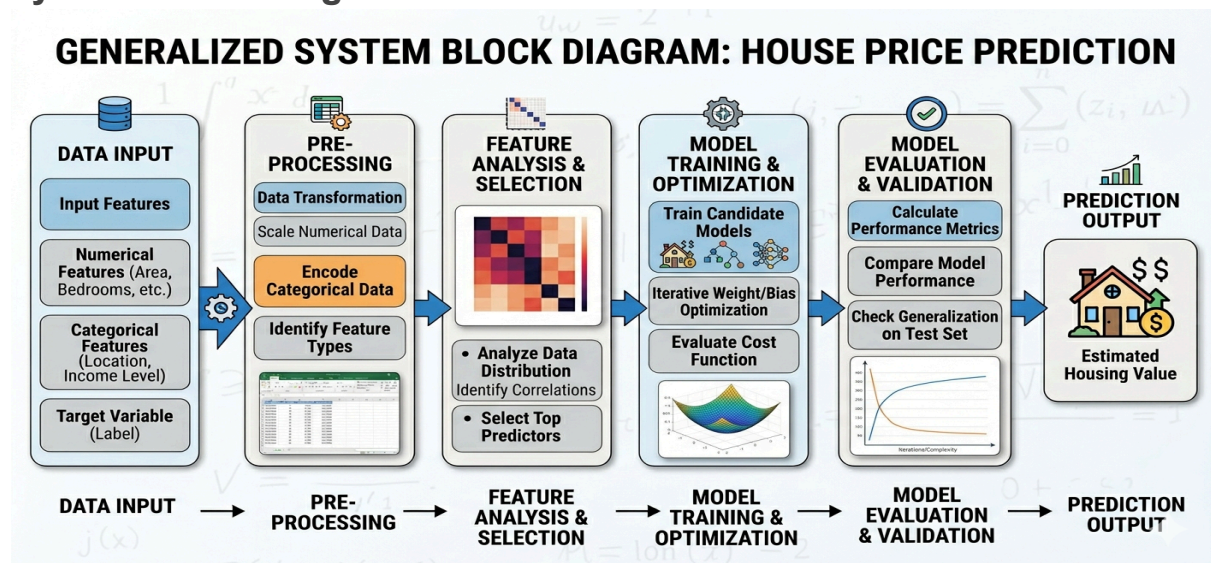
Problem Statement

The main aim of this report is to design and implement a computational model that will be able to predict the house price (y) based on a high dimensional feature vector (x). Given a dataset $D = \{(x(i), y(i))\}_{i=1}^N$ where $N = 50,000$, the goal is to minimize the cost function $J(w)$, representing the error between the predicted values and target values.

This involves:

1. **Feature Mapping:** Successfully transforming 18 heterogeneous features (numerical, categorical, and binary) into a normalized format for model consumption.
2. **Function Approximation:** Using Supervised Learning to approximate the complex, potentially non-linear function.
3. **Optimization:** Minimizing the cost function (Mean Squared Error) through the iterative update of Weights and Biases.

System Block Diagram:



*image generated using Google Gemini AI

Underlying Theory:

- **Linear Regression and Regularization:**

- Linear Regression serves as the fundamental supervised learning model. It assumes the target y (price) is a linear combination of the input features x :

$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \sum w_ix_i + b$$

- We apply regularization to ensure better generalization and prevent overfitting.

- **Artificial Neural Networks (ANN):**

- **Multi-layer Perceptron (MLP)** consists of interconnected layers of neurons. Each neuron in a **Hidden Layer** processes inputs using the following transformation:

$$z = \sum(w_ix_i) + \text{bias}$$

$$a = \sigma(z)$$

where σ is the Activation Function.

- **Backpropagation & Gradient Descent:**

- **Forward Pass:** The input signal passes forward through the neural network to produce an estimate of the price and its corresponding error (MSE).
- **Backward Pass:** The error is passed backward through the network. The partial derivative of the error function with respect to each weight is calculated as follows:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial E}{\partial w}$$

where η is the learning rate and E is the Mean Squared Error.

- **Decision Trees for Regression:**
 - A **Decision Tree** creates **Decision Boundaries** which are formed through recursive partitioning of the feature space. It selects the feature and the split point that results in the max reduction in variance for the target variable (price). This model is highly interpretable.

Dataset Description

Source

<https://www.kaggle.com/datasets/miadul/house-price-prediction-dataset>

Overview

This dataset is a realistic **synthetic** house price dataset designed to simulate real-world real estate scenarios. It contains **50,000** records with **18** features, capturing various aspects of residential properties such as size, location, amenities, and socio-economic factors.

Features Description

Category	Features	Description
Property Features	area, bedrooms, bathrooms, floors, age	Quantitative structural measurements and property history.
Location & Accessibility	distance, location, public_transport, school_nearby, hospital_nearby, shopping_mall_nearby	Geographic position relative to the city center and transport ease.
Amenities	garage, parking, garden, security	Binary indicators (0 or 1) representing the presence of modern facilities.

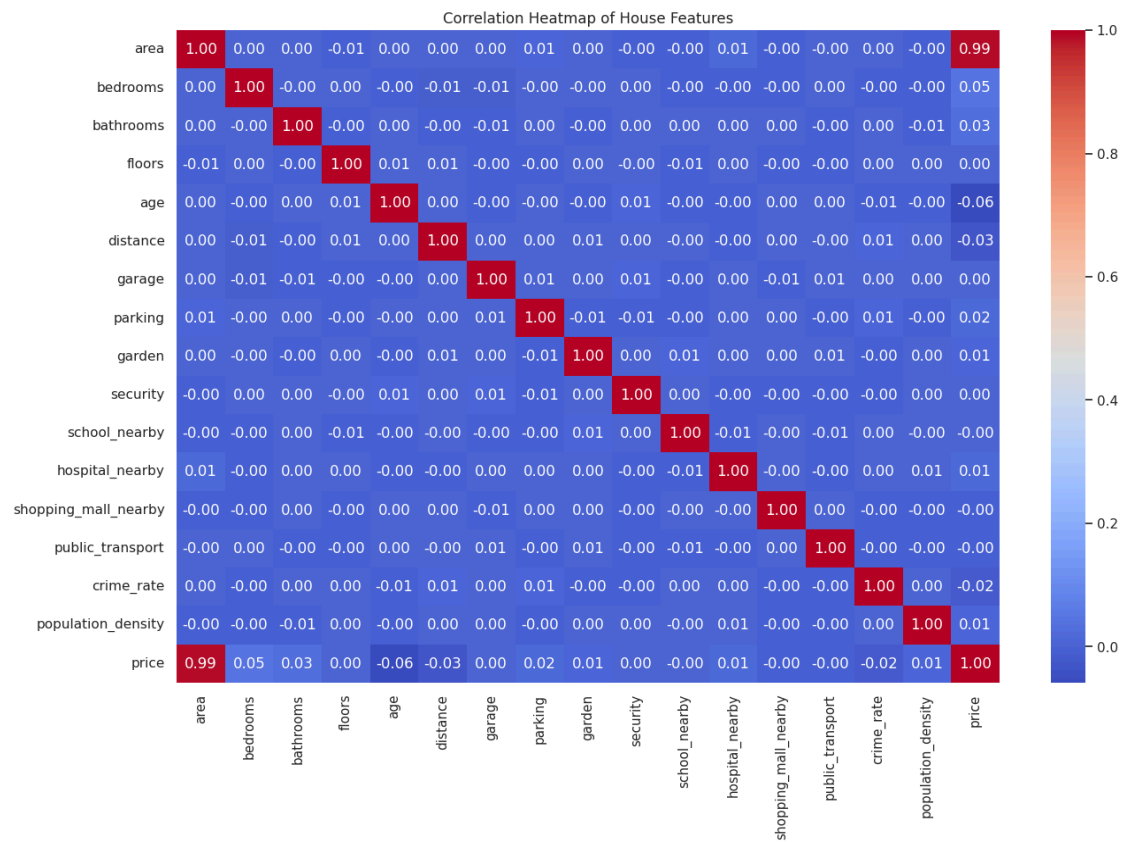
Socio-economic Factors	crime_rate, population_density, income_level	Environmental safety and economic status of the neighborhood.
------------------------	--	---

Target Variable

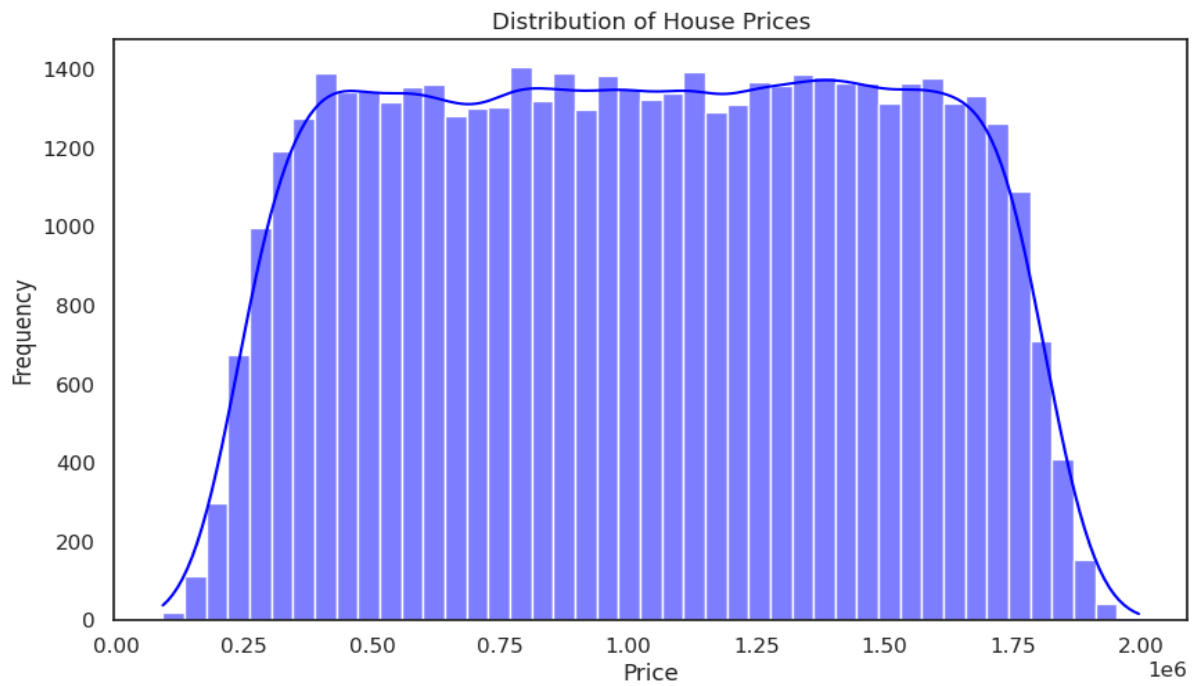
price: Estimated house price (generated using a realistic formula with noise)

Visualization

1. Correlation Heatmap



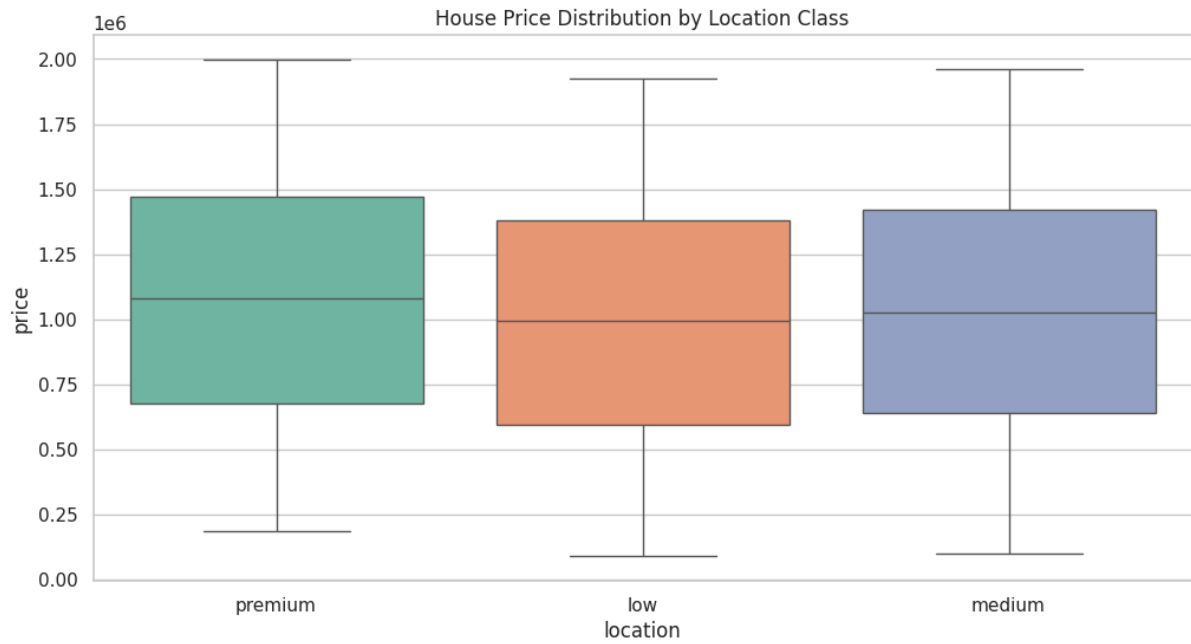
2. Distribution of House Prices



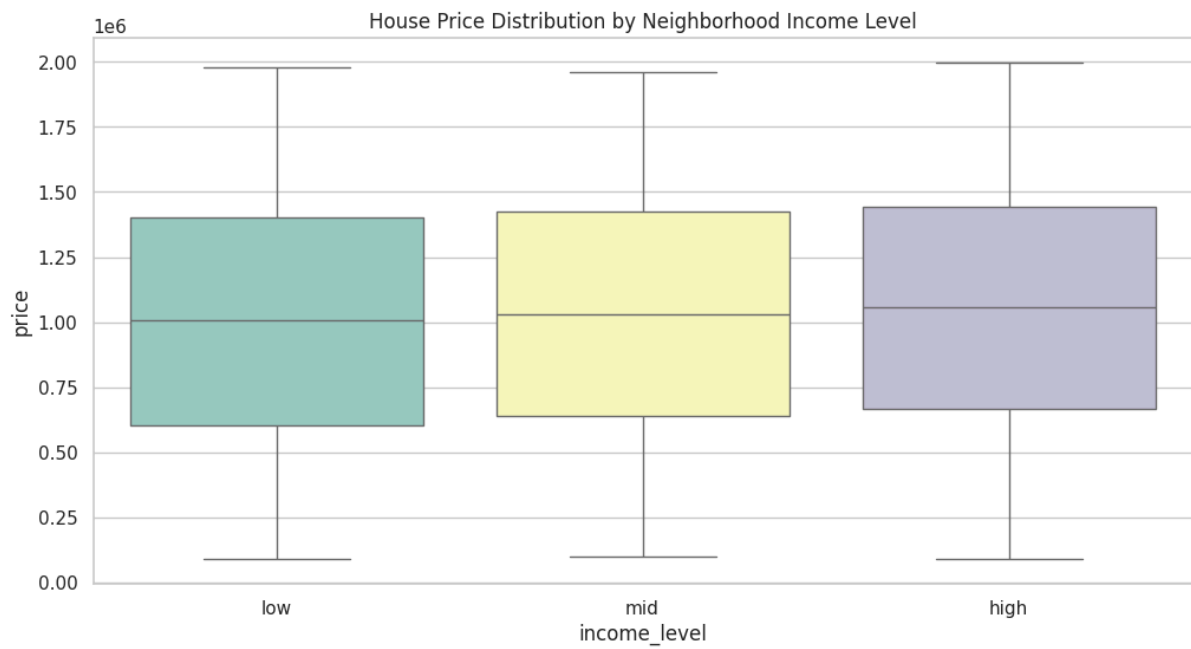
3. Price vs. Area



4. Location class



5. Neighborhood income class



Results

Performance Metrics Table

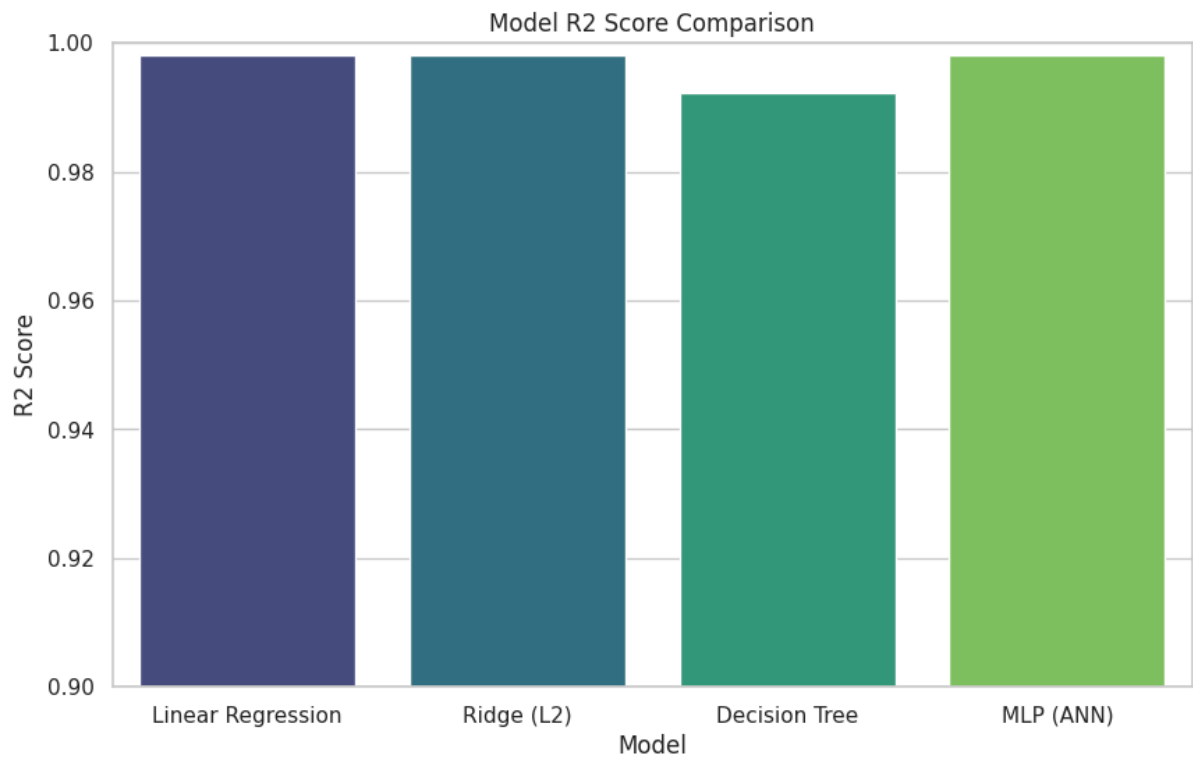
Model	MAE	RMSE	R2 Score
Linear Regression	15,954.17	19,940.84	0.99811
Ridge Regression (L2)	15,954.10	19,940.74	0.99811
Decision Tree	32,199.52	40,371.74	0.99226
MLP (ANN)	16,022.08	20,029.59	0.99809

Comparative Analysis

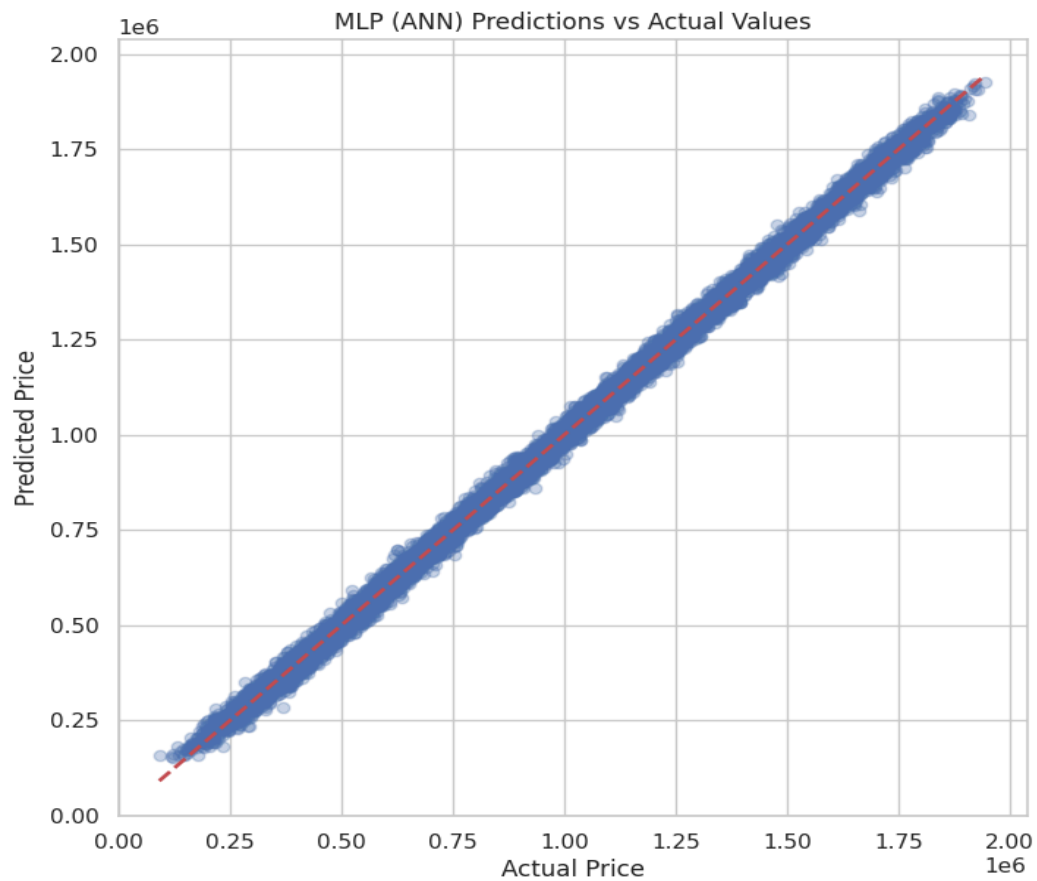
- Linear models vs Neural Networks:** The Linear Regression and Multi-layer Perceptron (MLP) models achieved nearly identical results, with R^2 scores exceeding 0.998. This means that the relation between features (like area and location) and price is predominantly linear in the dataset.
- Impact of Regularization:** The Ridge (L2) model showed a marginal (very less) improvement in MAE.
- Decision Tree Performance:** It showed the highest error rates. While it effectively creates Decision Boundaries by splitting data, its step-like prediction nature is less efficient than the continuous mapping of other models.

Visualizations

1. Model R^2 Comparison



2. Actual vs. Predicted Plot (MLP)



Discussion on Training Convergence

During the training of MLP (ANN), the loss function (MSE) decreased consistently over the iterations. The use of ReLU activation function helps the network to learn efficiently.

Conclusion

The project successfully implemented a high-accurate price prediction system using a dataset of 50K records. As we explored both classical statistical methods and adaptive neural networks, we gained significant insights into real estate valuation.

The key conclusions are as follows:

- **Model Accuracy:** Both Linear Regression and Multi-Layer Perceptron have achieved R^2 score of 0.998, which indicates that the selected features provide a nearly complete representation of the factors that drive the market value of the houses.
- **Neural Network Efficiency:** The implementation of Backpropagation algorithm and Gradient Descent optimization was highly effective. The MLP successfully mapped the 18-dimensional input to the target output, which shows that ANN is highly precise when properly tuned.
- **Generalization and Regularization:** The application of Ridge (L2) Regularization was effective in stabilising the model. By penalizing large Weights, we ensured that the models are generalized, preventing overfitting to the noise.
- **Interpretability:** Even though the decision tree provided a higher error rate, it provided valuable insights into how decision boundaries are formed based on categorical classes like income levels.

In summary, Regularised linear models and ANN offers a balance of speed and accuracy for large-scale datasets with strong linear characteristics. However, MLP is a superior choice as it is more flexible in complex, non-linear environments.

Future work

- This could involve using Principle Component Analysis (PCA) to reduce feature redundancy.
- Applying K-means Clustering to segment the housing market before training localized models.

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import drive

# Machine Learning imports
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_absolute_error, r2_score,
mean_squared_error
sns.set(style="white", color_codes=True)

drive.mount('/content/drive')
dataset = pd.read_csv('/content/drive/My Drive/Colab
Notebooks/datasets/house_price_50k.csv')
dataset.head()

# Select only numeric columns for correlation
numeric_df = dataset.select_dtypes(include=['float64', 'int64'])
sns.set(style="whitegrid")

plt.figure(figsize=(15, 10))
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap of House Features')
plt.show()

plt.figure(figsize=(10, 6))
sns.histplot(dataset['price'], kde=True, color='blue')
plt.title('Distribution of House Prices')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.show()

plt.figure(figsize=(10, 6))
sns.scatterplot(x='area', y='price', data=dataset, alpha=0.5)
plt.title('House Price vs. Total Area')
```

```

plt.xlabel('Area (sq ft)')
plt.ylabel('Price')
plt.show()

# Visualization of Location Classes
plt.figure(figsize=(12, 6))
sns.boxplot(x='location', y='price', data=dataset, palette='Set2',
hue='location', legend=False)
plt.title('House Price Distribution by Location Class')
plt.show()

# Visualization of Income Level Classes
plt.figure(figsize=(12, 6))
sns.boxplot(x='income_level', y='price', data=dataset, palette='Set3',
hue='income_level', legend=False)
plt.title('House Price Distribution by Neighborhood Income Level')
plt.show()

# Define features and target
X = dataset.drop('price', axis=1)
y = dataset['price']

# Identify column types
categorical_features = ['location', 'income_level']
numerical_features =
X.select_dtypes(include=[np.number]).columns.tolist()

# Create a preprocessor
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(), categorical_features)
    ])

# Split the data (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

print("Preprocessing complete. Data split into Train/Test sets.")

# 1. Linear Regression (Baseline)
lr_model = Pipeline(steps=[('preprocessor', preprocessor),
('regressor', LinearRegression())])

```

```

# 2. Ridge Regression (L2 Regularization)
ridge_model = Pipeline(steps=[('preprocessor', preprocessor),
                              ('regressor', Ridge(alpha=1.0))])

# 3. Decision Tree Regressor
dt_model = Pipeline(steps=[('preprocessor', preprocessor),
                              ('regressor', DecisionTreeRegressor(max_depth=10, random_state=42))])

# 4. Multi-layer Perceptron (ANN)
# Uses ReLU activation and Adam optimizer (Gradient Descent variant)
mlp_model = Pipeline(steps=[('preprocessor',
                              preprocessor), ('regressor', MLPRegressor(hidden_layer_sizes=(64, 32),
                              activation='relu', solver='adam', max_iter=500, random_state=42))])

# Train all models
models = {
    "Linear Regression": lr_model,
    "Ridge (L2)": ridge_model,
    "Decision Tree": dt_model,
    "MLP (ANN)": mlp_model
}

for name, model in models.items():
    print(f"Training {name}...")
    model.fit(X_train, y_train)

print("All models trained successfully.")

results = []

for name, model in models.items():
    predictions = model.predict(X_test)
    mae = mean_absolute_error(y_test, predictions)
    r2 = r2_score(y_test, predictions)
    rmse = np.sqrt(mean_squared_error(y_test, predictions))
    results.append({"Model": name, "MAE": mae, "R2 Score": r2, "RMSE":
rmse})

# Display Results Table
results_df = pd.DataFrame(results)
print("\nModel Performance Summary:")
print(results_df)

```

```
# Visualization: R2 Score Comparison
plt.figure(figsize=(10, 6))
sns.barplot(x='Model', y='R2 Score', data=results_df,
palette='viridis', hue='Model', legend=False)
plt.title('Model R2 Score Comparison')
plt.ylim(0.9, 1.0) # Zoom in to see differences
plt.show()

# Visualization: Actual vs Predicted (for ANN)
y_pred_mlp = mlp_model.predict(X_test)
plt.figure(figsize=(8, 8))
plt.scatter(y_test, y_pred_mlp, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
'r--', lw=2)
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')
plt.title('MLP (ANN) Predictions vs Actual Values')
plt.show()
```

Submitted by:

Name: Kona Bhuvaneswar Reddy

Roll No: B240214CS

Branch: Computer Science and Engineering

Date of Submission: 20th April, 2026